

# phorce 퀵 가이드

젯슨을 처음 받은 여러분이 **10 분 안에 로봇을 한 번 움직여 보는 것**까지. 코드는 한 줄도 안 씁니다.  
순서대로만 따라오세요.

## 1. 이게 뭔가요? (30 초)

여러분 앞에는 세 가지가 있습니다. 사람 몸에 비유하면 이렇습니다.

| 이름          | 정체                     | 몸에 비유하면         |
|-------------|------------------------|-----------------|
| Jetson (젯슨) | 작은 컴퓨터. 여러분이 코드를 돌리는 곳 | 뇌 — 생각하고 명령을 내림 |
| pcm         | Jetson 과 모터 사이의 중계기    | 신경 — 명령을 전달     |
| phact       | 관절마다 붙은 모터 장치          | 근육 — 실제로 움직임    |

여러분은 **뇌(Jetson)** 에서 일합니다. 세 개를 케이블로 연결하고 뇌에게 "이 동작 해"라고 말하면, 신경(pcm)을 타고 근육(phact)이 움직입니다.

## 2. 딱 두 가지만 기억하세요

이 시스템에서 여러분이 하는 일은 **단 두 방향**뿐입니다. 어렵게 생각하지 마세요.

↑ 올라오는 것 — 로봇이 지금 어떤지 (1 초에 1000 번!)

로봇은 관절들의 위치·속도·힘·온도를 **1 초에 1000 번** 여러분에게 계속 알려줍니다. 이걸

/phorce/feedback 이라고 부릅니다. 여러분의 눈입니다. (피드백 칸은 12 개지만 현재 로봇에 장착된 관절은

6 개 — 빈 칸은 구분할 수 있게 표시되어 있습니다.)

↓ 내려가는 것 — "몇 번 동작 해줘"

여러분은 미리 저장된 동작 중 하나를 번호(1~50)로 고릅니다. "7 번 동작 재생!" 이렇게요. 로봇이 그

동작을 합니다. 여러분의 입입니다.

즉, 로봇 상태를 보고(↑) → 몇 번 동작을 할지 정해서 보낸다(↓). 이게 전부입니다. 관절 하나하나를 직접 조종하는 게 아니라, 이미 만들어진 동작을 골라 트는 겁니다 (노래방에서 곡 번호 고르는 것과 똑같아요).

### 3. 시작 전 확인, 그리고 시동 걸기 (3 분)

#### 3-1. 확인 세 가지

- 젯슨 전원이 켜져 있고, 화면에 로그인 창이 보이나요?
- 처음 로그인하면 비밀번호를 새로 정하라고 나옵니다 — 정상입니다. 새 비밀번호를 정하세요.
- 로봇(pcm·phact) 전원을 먼저 켜세요. 케이블 연결 확인은 터미널에서 한 줄:

```
cat /sys/class/net/eno1/operstate # "up" 이 나오면 로봇과 연결된 것
```

down 이 나오면 로봇 전원과 eno1 케이블을 확인하세요.

### 3-2. 시동 걸기 — 터미널 2 개

로봇과 대화하려면 여러분의 젓손에서 **두 개의 프로그램**이 돌고 있어야 합니다. 터미널을 두 개 열고 하나씩 실행하세요 (**켜 둔 채로 두는 겁니다** — 닫으면 연결이 끊깁니다):

```
# 터미널 1 — 로봇 통신 담당 (13 개 자가검사가 전부 PASS 로 지나가야 정상)
```

```
ros2 run agx_phorce_bridge phorce_monitor --ros-args -p nic:=eno1 -p mode:=command -p axes:=2 -p  
mbx_enabled:=true
```

```
# 터미널 2 — 모션 요청 창구
```

```
ros2 run agx_motion_slot motion_action_server --ros-args -p backend:=ecat
```

**순서 규칙:** 반드시 **로봇 전원이 먼저**, 터미널 1 이 그 다음입니다. 로봇을 켜다 켜다면 터미널 1 도 **Ctrl+C** 후 다시 실행하세요 — 로봇 정보를 시작할 때 한 번 읽기 때문입니다.

이제 **세 번째 터미널**을 열어 아래 4 장을 진행합니다.

## 4. 10 분 따라하기

### 1로봇과 대화가 되는지 확인

```
phorce doctor
```

로봇 시스템이 살아있는지 건강검진하는 명령입니다. 초록색 **PASS** 나 **ok** 가 주르륵 나오면 통과입니다.

**안 되면?** **command not found** 가 나오면 로그아웃했다가 다시 로그인해 보세요. 그래도 안 되면 운영진에게 알려주세요.

### 2로봇 상태가 올라오는지 눈으로 보기

```
ros2 topic hz /phorce/feedback
```

**1000** 근처 숫자가 계속 나오면, 로봇이 1 초에 1000 번 자기 상태를 보내는 중입니다. 살아있다는 증거예요. (

멈추려면 **Ctrl + C**)

### 3 지금 할 수 있는 동작 목록 보기

phorce list

재생할 수 있는 동작이 번호와 이름으로 나옵니다. 예: 01 손흔들기. **여기 나오는 번호만** 재생할 수 있어요.

**왜 목록이 로봇에 있나요?** 이 목록은 젯슨이 아니라 **로봇이 실제로 품고 있는 동작**입니다. 로봇에 저장 안 된 동작은 목록에 안 뜨고, 재생도 안 됩니다. **목록에 뜨면 = 쓸 수 있다** 로 기억하세요.

### 4 로봇을 "준비" 상태로 (사람이 버튼)

전원만 켜졌다고 로봇이 동작을 받지는 않습니다. **기체의 영점 버튼(1 번)**을 **0.6 초 이상** 누르세요.

- 로봇이 **영점 자세**를 잡습니다 → **움직입니다. 주변을 먼저 확인!**
- 약 **3 초** 뒤 서보가 켜지고 **동작 받을 준비**가 됩니다.

**이 버튼을 안 누르면** 다음 단계에서 "아직 준비 안 됨 (REJECT\_NOT\_READY\_FOR\_MOTION)"으로 거절당합니다. **기다려도 안 풀립니다 — 사람이 눌러야 합니다.**

### 5 첫 동작 재생 — 코드 0 줄

phorce play 1

**1 번 동작이 실행됩니다.** 진행률이 표시되고, 끝나면 완료 메시지가 나옵니다. 축하합니다 — 여러분이 방금 로봇을 움직였어요! (다른 번호도 **phorce play 3** 처럼)

**실수를 겁내지 않아도 됩니다.** 없는 번호를 보내거나 준비 안 된 상태에서 보내면 로봇이 **스스로 거절**하고 이유를 알려 줍니다. 위급할 때는 언제나 **물리 E-Stop 버튼**입니다.

## 5. 버튼으로 하고 싶다면 — 화면(GUI)

phorce-console

로봇 상태를 **그래프로 보고**, 동작을 **버튼으로 쏘는** 화면이 뜹니다. 명령어가 어색하면 이걸 쓰세요. 켜자마자 할 일 하나: 화면 위쪽 **대상** 칸을 ● **실물 로봇**으로 바꾸세요 — 그러면 테두리가 **빨개지고** 버튼이 **[실물 전송]**으로 바뀝니다. (기본값으로 선택된 "시뮬레이터"는 이 행사에서는 쓰지 않습니다 — 그대로 두면 로봇이 안 움직입니다.) 누르기 전에 항상 주변을 확인하세요.

## 6. 안전 — 이것만은 꼭

| 상황          | 어떻게                                               |
|-------------|---------------------------------------------------|
| 비상 정지       | 물리 <b>E-Stop</b> 버튼이 유일합니다. 위급하면 그걸 누르세요 (키보드 아님) |
| 실물 전송 전     | 항상 <b>로봇 주변에 사람·물건이 없는지</b> 확인                    |
| "준비 안 됨" 거절 | 영점 버튼(1 번) 0.6 초 누르고 3 초 기다리기                     |
| "복구 필요" 거절  | 2 번 버튼으로 파킹 → 다시 영점 버튼                            |

## 7. 다음은?

여기까지 왔다면 로봇을 켜고 움직이는 법을 익힌 겁니다. 이제 **여러분의 코드로** "로봇을 보고 스스로 판단해 동작을 고르게" 만들 차례입니다.

- ② **튜토리얼** ([02-tutorial](#)) — 파이썬/C++ 로 로봇 상태를 읽고, 그에 따라 동작을 고르는 법을 한 단계씩
- ③ **매뉴얼** ([03-manual](#)) — 규칙·안전·용어 참조표
- ④ **API 레퍼런스** ([04-api-reference](#)) — 함수·필드 하나하나의 사전. 코딩하다 막히면 여기
- ⑤ **시스템 안내** ([05-system-image](#)) — 이 젓슨이 어떻게 구성돼 있는지 (리눅스에 익숙하다면)